FITLIFE · PROJECT RECORD

Project Work Log

A chronological record of implementation, troubleshooting, verification, and documentation

Project: Flutter fitness-centre app Work period: 13 October 2026 Platform: Windows

> **Outcome**
> The app's provider syntax was corrected, regression coverage was added, compact-card layout overflows were addressed, and the generated counter test was removed. The last automated checks reported `flutter test` passing and `flutter analyze` with no issues. Windows and browser launch attempts exposed local tooling problems along the way; the user subsequently confirmed that the app is now working.

This report records the meaningful steps taken in the FitLife session, including unsuccessful attempts and the evidence they produced. Commands are shown as run or as the equivalent project-root command when several retries differed only in shell/path syntax.

Project at the Start

FitLife was an existing Flutter app scaffold with Material 3 UI, Riverpod state providers, GoRouter navigation, feature screens, reusable cards, and local sample data. The main areas were Home, Programs, Trainers, Membership, and Profile, with separate program and trainer detail routes. No backend service was part of the current prototype.

The first status review found the app had substantial UI and routing already present, but it had not yet been verified end-to-end. A suspected syntax problem in the membership provider and uncertain analyzer results became the first concrete checks.

Chronology

1. Inspected the app and checked its code

Reviewed the app entry point, provider layer, router, mock data, project dependencies, and feature screens. Ran Flutter analysis and tests to establish a baseline. Early analyzer execution timed out, so the result was treated as inconclusive rather than a pass.

Finding: `membershipsProvider` returned `MockData.memberships` without a terminating semicolon. The malformed provider was a concrete compile blocker.

2. Fixed the provider and added regression coverage

Added the missing semicolon in `lib/providers/app_providers.dart`. Created `test/app_providers_test.dart` to verify that providers expose program, trainer, and membership data and that featured lists remain bounded.

Check: `flutter test` passed with one test. This established that the provider layer compiled and returned the expected mock data.

3. Tried browser launch and diagnosed the local port conflict

Attempted to run the app in Chrome and via Flutter's web-server target, initially on port 3000. The launch could not establish a usable debug session. A later direct Chrome attempt reported that port 3000 was already bound (Windows socket error 10048); process inspection pointed to a leftover Dart/Flutter debug process. This evidence indicated a local port conflict rather than a firewall rule.

Checked available Flutter devices and confirmed Windows, Chrome, and Edge were detected. Retried the web launch on other ports and in Edge. Edge reached "Waiting for connection from debug service" but did not yield a stable verified preview. A Flutter temp-directory cleanup permission/access error was also observed.

4. Prepared the Windows desktop target

The first native run reported that the project had no Windows desktop configuration. Enabled Flutter Windows desktop support and generated the Windows runner files with `flutter config --enable-windows-desktop` and `flutter create . --platforms=windows`.

Windows Developer settings were opened after Flutter indicated symlink/Developer Mode requirements for plugin builds. A later Windows build completed and produced `build/windows/x64/runner/Debug/fitlife.exe`, confirming the desktop target could compile.

5. Used runtime rendering to find card overflows

Running the app far enough to lay out the UI exposed a trainer-card `RenderFlex` overflow of 28 pixels. Follow-up widget testing surfaced another 12-pixel overflow in the horizontal program card, as well as the trainer card overflow under test constraints.

Adjusted `lib/widgets/trainer_card.dart` to use a fixed card height and more compact avatar, text, and spacing. Adjusted the horizontal layout in `lib/widgets/program_card.dart` to prevent the card's constrained column from overflowing. These changes addressed the reported compact-card layout failures.

6. Removed the stale generated counter test

The scaffold's default `test/widget_test.dart` expected a `MyApp` counter screen, which does not exist in FitLife. It was changed temporarily to pump the actual `FitLifeApp`, but the app's timed entrance animations made `pumpAndSettle` time out and also exposed the card layout overflows. Rather than keep an unreliable generated test, the stale widget test was removed. The meaningful provider regression test remained.

7. Reset Flutter's local build/debug state

Stopped stale Dart/Flutter processes and processes holding common development ports, removed stale Flutter temp debug folders, then ran `flutter clean`, `flutter pub get`, and `flutter analyze`. The intent was to eliminate stale tooling state and revalidate from a clean dependency/build state.

8. Re-ran automated checks and attempted final launch

The project checks were run after the layout and test cleanup:

- `flutter test` → **All tests passed** (one provider regression test).
- `flutter analyze` → **No issues found.**

A final `flutter run -d windows` attempt began building the Windows application but did not, at that point, provide a confirmed visible UI in the session. Web launch attempts had separately encountered the occupied port/debug-service issue. These were recorded as environment/runtime launch friction, not as passing visual verification.

Final user confirmation: after these changes and launch attempts, the user reported: "the app is now working thanks." This is the final confirmation of working behavior.

Documentation Added

After confirming the app was working, the project's placeholder `README.md` was replaced with project-specific documentation. It now covers app features, prerequisites, dependency setup, Chrome and Windows run commands, test and analyzer commands, folder layout, remote sample images, and prototype limitations.

Current Verification Snapshot

Area	Recorded result
Provider compile blocker	Fixed: membership provider return statement now terminates correctly.
Regression test	Added for mock provider data and featured-list bounds; the retained suite passed.
Static analysis	`flutter analyze` reported no issues in the final code checks.
Card layout	Trainer and horizontal program-card overflow reports were addressed.
Windows target	Desktop files generated and a Windows debug executable built.
App running	User confirmed the app was working after the launch troubleshooting.
Backend/persistence	Not implemented in this prototype. Sample data and app state are local/in-memory.

Useful Commands

Purpose	Command
Install dependencies	`flutter pub get`
Run tests	`flutter test`
Analyze source	`flutter analyze`
Run in Chrome	`flutter run -d chrome`
Run as Windows desktop	`flutter run -d windows`

This is a session work log, not a claim that every platform, screen size, or external service has been exhaustively tested. Sample images use remote Unsplash URLs and require an internet connection.